

베이지안 기계학습

Dirichlet Processes

조성일

인하대학교
통계학과

Table of contents

1. Introduction
2. Dirichlet Process
3. Dirichlet Process Mixture Models
4. Examples
 - ▶ Clustering
 - ▶ Density estimation

Introduction

Motivation: mixture model I

- To motivate the Dirichlet process, let us consider a simple density estimation problem: modeling the height of students.

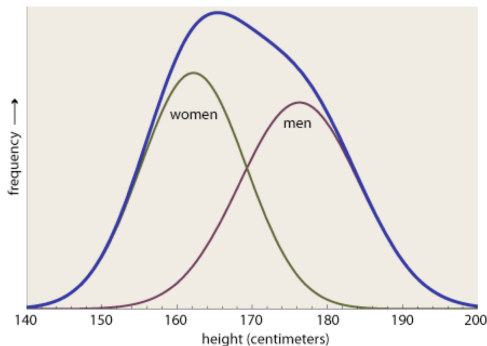


Figure: <https://www.stat.ubc.ca/~bouchard/courses/stat547-sp2011/notes-part2.pdf>

Motivation: mixture model II

We observe $y_1, \dots, y_n$ with $y_i | F \stackrel{iid}{\sim} F$.

- ▶ Finite mixture model is a popular choice for density estimation,

$$F(y) = \sum_{h=1}^K w_h g(y; \theta_h)$$

- ▶ It is well known that a mixture of normals can approximate any density.
- ▶ Do we believe that the true density is a mixture of K components?
- ▶ How many components to use?

Motivation: mixture model II

We observe $y_1, \dots, y_n$ with $y_i | F \stackrel{iid}{\sim} F$.

- ▶ Finite mixture model is a popular choice for density estimation,

$$F(y) = \sum_{h=1}^K w_h g(y; \theta_h)$$

- ▶ It is well known that a mixture of normals can approximate any density.
- ▶ Do we believe that the true density is a mixture of K components?
- ▶ How many components to use?

Nonparametric priors !!

Dirichlet process

Dirichlet process (Ferguson, 1973)

- ▶ A Dirichlet Process (DP) is defined as a distribution over random probability measures.

Dirichlet process (Ferguson, 1973)

- ▶ A Dirichlet Process (DP) is defined as a distribution over random probability measures.
- ▶ We write:

$$G \sim DP(\alpha_0, G_0)$$

if for any finite measurable partition $(A_1, \dots, A_r)$ of Θ ,

$$(G(A_1), \dots, G(A_r)) \sim \text{Dirichlet}(\alpha_0 G_0(A_1), \dots, \alpha_0 G_0(A_r)),$$

where G_0 is a specified probability measure and α_0 is a positive scalar parameter.

Dirichlet process (Ferguson, 1973)

- ▶ A Dirichlet Process (DP) is defined as a distribution over random probability measures.
- ▶ We write:

$$G \sim DP(\alpha_0, G_0)$$

if for any finite measurable partition $(A_1, \dots, A_r)$ of Θ ,

$$(G(A_1), \dots, G(A_r)) \sim \text{Dirichlet}(\alpha_0 G_0(A_1), \dots, \alpha_0 G_0(A_r)),$$

where G_0 is a specified probability measure and α_0 is a positive scalar parameter.

- ▶ **Remark.** The existence of the DP can be showed by Kolmogorov consistency theorem, and de Finetti's theorem.

Dirichlet process: Stick-breaking construction

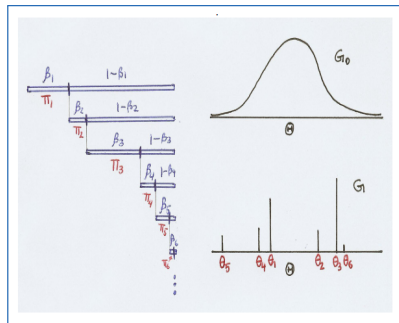
- Sathuraman (1994) showed that

$$G = \sum_{k=1}^{\infty} \pi_k \delta_{\theta_k} \sim DP(\alpha_0, G_0) \text{ if}$$

$$\pi_k = \beta_k \prod_{l=1}^{k-1} (1 - \beta_l),$$

$$\beta_l \sim \text{Beta}(1, \alpha_0),$$

where $\theta_k \sim G_0$.



Dirichlet process: Stick-breaking construction

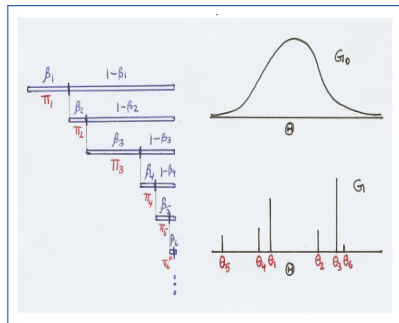
- Sathuraman (1994) showed that

$$G = \sum_{k=1}^{\infty} \pi_k \delta_{\theta_k} \sim DP(\alpha_0, G_0) \text{ if}$$

$$\pi_k = \beta_k \prod_{l=1}^{k-1} (1 - \beta_l),$$

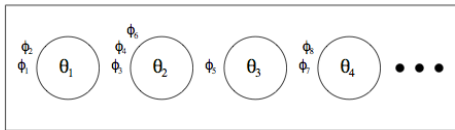
$$\beta_l \sim \text{Beta}(1, \alpha_0),$$

where $\theta_k \sim G_0$.



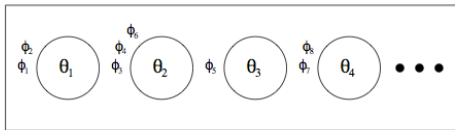
- $\sum_{k=1}^{\infty} \pi_k = 1$ with probability 1 (Ishwaran & James, 2001).
- Samples drawn from a DP are discrete with probability 1.
- We write $\pi \sim GEM(\alpha_0)$ if $\pi = (\pi_1, \pi_2, \dots)$ is defined as above.

Dirichlet process: Chinese restaurant process (Aldous, 1985)



- Consider a Chinese restaurant with an unbounded number of tables and seats, respectively.

Dirichlet process: Chinese restaurant process (Aldous, 1985)



- ▶ Consider a Chinese restaurant with an unbounded number of tables and seats, respectively.
- ▶ The i th customer, ϕ_i , sits down
 - ▶ at the occupied table indexed by θ_k with probability $\frac{m_k}{\alpha_0 + i - 1}$,
 - ▶ at a new table with probability $\frac{\alpha_0}{\alpha_0 + i - 1}$.

where $m_k = \#\{i' : \phi_{i'} = \theta_k \text{ for } 1 \leq i' < i\}$.

Dirichlet process: properties I

- (Interpretability) If $G \sim DP(\alpha_0, G_0)$, then for any measurable A , $G(A) \sim \text{Beta}(\alpha_0 G_0(A), \alpha_0 G_0(A^c))$, and thus

$$\mathbb{E}(G(A)) = G_0(A) \quad \text{and} \quad \mathbb{V}(G(A)) = \frac{G_0(A)\{1 - G_0(A)\}}{\alpha_0 + 1}.$$

Dirichlet process: properties I

- (Interpretability) If $G \sim DP(\alpha_0, G_0)$, then for any measurable A , $G(A) \sim \text{Beta}(\alpha_0 G_0(A), \alpha_0 G_0(A^c))$, and thus

$$\mathbb{E}(G(A)) = G_0(A) \quad \text{and} \quad \mathbb{V}(G(A)) = \frac{G_0(A)\{1 - G_0(A)\}}{\alpha_0 + 1}.$$

- G_0 plays the role of the *center* of the DP (referred to as base measure or base distribution)
- α_0 can be viewed as a precision parameter: for large α_0 there is small variability in DP realizations; the larger α_0 is, the *closer* we expect a realization G from the process to be to G_0 .

Dirichlet process: properties II

$G \sim DP(\alpha_0, G_0)$ and $Y_1, \dots, Y_n | G \sim G$.

Then,

- (Conjugacy) $G | Y_1, \dots, Y_n \sim DP\left(n + \alpha_0, \frac{\alpha_0 G_0 + \sum_{i=1}^n \delta_{Y_i}}{\alpha_0 + n}\right)$.

Dirichlet process: properties II

$G \sim DP(\alpha_0, G_0)$ and $Y_1, \dots, Y_n | G \sim G$.

Then,

- ▶ (Conjugacy) $G | Y_1, \dots, Y_n \sim DP\left(n + \alpha_0, \frac{\alpha_0 G_0 + \sum_{i=1}^n \delta_{Y_i}}{\alpha_0 + n}\right)$.
- ▶ (Marginalization Property)
(Blackwell and MacQueen, 1973)
marginally $(Y_1, Y_2, \dots)$ forms a Pólya urn sequence:

$$Y_1 \sim G_0$$

$$Y_{n+1} | Y_1, \dots, Y_n \sim \frac{\alpha_0}{\alpha_0 + n} G_0 + \frac{1}{\alpha_0 + n} \sum_{i=1}^n \delta_{Y_i}, \quad n \geq 1.$$

Dirichlet Process Mixture Models

Dirichlet Process Mixture Models

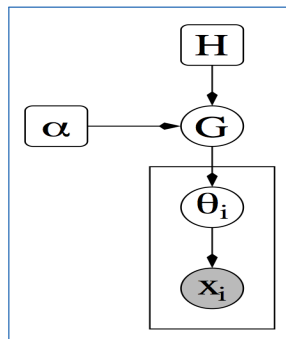
- ▶ A DP mixture model has the following form

$$x_i | \phi_i \sim F(\phi_i), \quad i = 1, \dots, n$$

$$\phi_i | G \sim G,$$

$$G \sim DP(\alpha_0, G_0),$$

where $F(\phi_i)$ is the distribution of x_i given ϕ_i .



Dirichlet Process Mixture Models

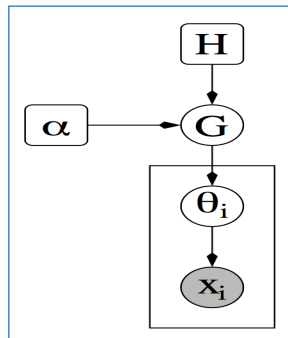
- ▶ A DP mixture model has the following form

$$x_i | \phi_i \sim F(\phi_i), \quad i = 1, \dots, n$$

$$\phi_i | G \sim G,$$

$$G \sim DP(\alpha_0, G_0),$$

where $F(\phi_i)$ is the distribution of x_i given ϕ_i .



- ▶ An equivalent representation is

$$\begin{aligned} \pi | \alpha_0 &\sim GEM(\alpha_0), & z_i | \pi &\sim \pi, \\ \theta_k | G_0 &\sim G_0, & x_i | z_i, (\theta_k)_{k=1}^\infty &\sim F(\theta_{z_i}). \end{aligned}$$

Moreover, $G = \sum_{k=1}^\infty \pi_k \delta_{\theta_k}$ and $\phi_i = \theta_{z_i}$.

Density estimation using DPMM

- ▶ DP mixture model is given by

$$x_1, x_2, \dots, x_n \stackrel{iid}{\sim} F(x) = \int g(x | \phi) G(d\phi), \quad G \sim DP(\alpha G_0),$$

where kernel $g(x | \phi)$ that yields mixtures with flexible density shapes is needed.

Density estimation using DPMM

- ▶ DP mixture model is given by

$$x_1, x_2, \dots, x_n \stackrel{iid}{\sim} F(x) = \int g(x | \phi) G(d\phi), \quad G \sim DP(\alpha G_0),$$

where kernel $g(x | \phi)$ that yields mixtures with flexible density shapes is needed.

- ▶ Infinite dimensional mixture model:

$$F(x) = \sum_{k=1}^{\infty} \pi_k g(x; \phi_k).$$

Examples

- ▶ Clustering
- ▶ Density estimation

Libraries

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tqdm import tqdm

import torch
import torch.nn.functional as F
from torch.distributions import constraints

import pyro
from pyro.distributions import *
from pyro.infer import Predictive, SVI, Trace_ELBO
from pyro.optim import Adam

pyro.set_rng_seed(0)
```

Example: Clustering

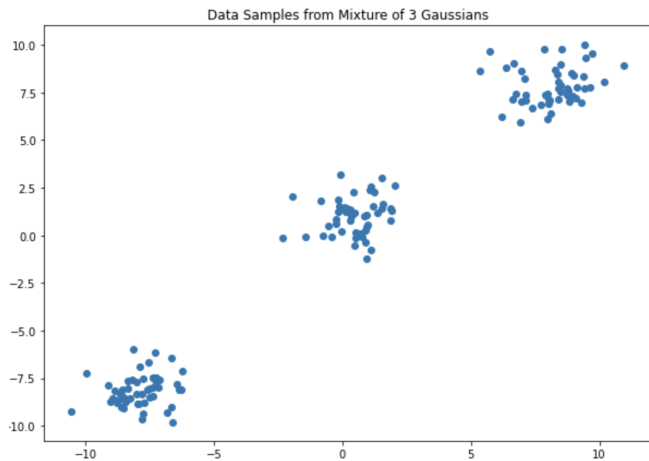
Clustering: dataset I

- ▶ A synthetic dataset generated by a mixture of three 2D Gaussians

```
MEAN = torch.ones(2)
COV = torch.eye(2)
MEAN2 = torch.tensor([0.5, 1])
data = torch.cat((MultivariateNormal(-8 * MEAN, COV).sample([50]),
                                     MultivariateNormal(8 * MEAN, COV).sample([50]),
                                     MultivariateNormal(MEAN2, COV).sample([50])))

plt.scatter(data[:, 0], data[:, 1])
plt.title("Data Samples from Mixture of 3 Gaussians")
plt.show()
N = data.shape[0]
```

Clustering: dataset II



Clustering: Dirichlet process mixture model

► Stick-breaking weights

```
def mix_weights(beta):  
    beta1m_cumprod = (1 - beta).cumprod(-1)  
    return F.pad(beta, (0,1), value=1)*F.pad(beta1m_cumprod, (1,0), value=1)
```

► DPMM

```
def model(data):  
    with pyro.plate("beta_plate", T-1):  
        beta = pyro.sample("beta", Beta(1, alpha))  
  
    I2 = torch.eye(2)  
    with pyro.plate("mu_plate", T):  
        mu = pyro.sample("mu", MultivariateNormal(torch.zeros(2), 5*I2))  
  
    with pyro.plate("data", N):  
        z = pyro.sample("z", Categorical(mix_weights(beta)))  
        pyro.sample("obs", MultivariateNormal(mu[z], I2), obs=data)
```

Clustering: variational distribution

► variational distribution

$$q(\beta, \theta, z) = \prod_{t=1}^{T-1} q_t(\beta_t) \prod_{t=1}^T q_t(\theta_t) \prod_{n=1}^N q_n(z_n)$$

```
def guide(data):
    kappa = pyro.param('kappa', lambda: Uniform(0, 2).sample([T-1]),
                        constraint=constraints.positive)

    I2 = torch.eye(2)
    tau=pyro.param('tau',lambda: MultivariateNormal(torch.zeros(2),3*I2).sample([T]))
    phi = pyro.param('phi',lambda: Dirichlet(1/T * torch.ones(T)).sample([N]),
                    constraint=constraints.simplex)

    with pyro.plate("beta_plate", T-1):
        q_beta = pyro.sample("beta", Beta(torch.ones(T-1), kappa))

    with pyro.plate("mu_plate", T):
        q_mu = pyro.sample("mu", MultivariateNormal(tau, torch.eye(2)))

    with pyro.plate("data", N):
        z = pyro.sample("z", Categorical(phi))
```

Clustering: model fitting I

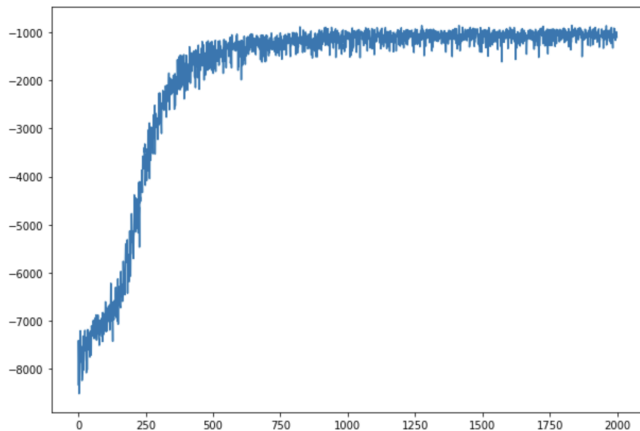
► Stochastic variational inference

```
T = 6
optim = Adam({"lr": 0.05})
svi = SVI(model, guide, optim, loss=Trace_ELBO())
elbo_values = []

def train(num_iterations):
    pyro.clear_param_store()
    for j in tqdm(range(num_iterations)):
        elbo = svi.step(data)
        elbo_values.append(-elbo)

alpha = 1
train(2000)
```

Clustering: model fitting II

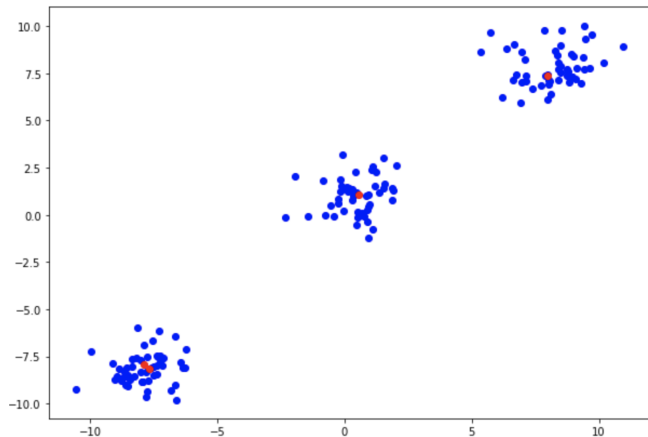


Clustering: model fitting III

► Posterior inference

```
def truncate(alpha, centers, weights):  
    threshold = alpha**-1/10.  
    true_centers = centers[weights > threshold]  
    true_weights = weights[weights > threshold] /  
        torch.sum(weights[weights > threshold])  
    return true_centers, true_weights  
  
# We make a point-estimate of our model parameters using  
# the posterior means of tau and phi for the centers and weights  
Bayes_Centers, Bayes_Weights = truncate(alpha, pyro.param("tau").detach(),  
                                         torch.mean(pyro.param("phi").detach(), dim=0))  
  
plt.figure(figsize=(10, 7))  
plt.scatter(data[:, 0], data[:, 1], color="blue")  
plt.scatter(Bayes_Centers[:, 0], Bayes_Centers[:, 1], color="red")
```

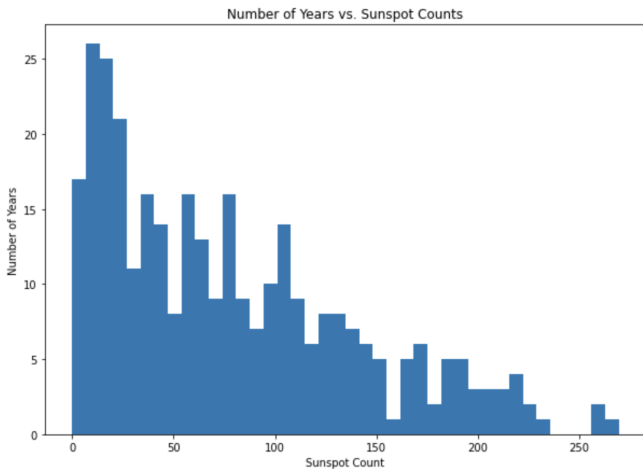
Clustering: model fitting IV



Example: Density estimation

Density estimation: Long term solar observations

► http://www.sidc.be/silso/DATA/SN_y_tot_V2.0.csv



Density estimation: Dirichlet process mixture model

► Stick-breaking weights

```
def mix_weights(beta):  
    beta1m_cumprod = (1 - beta).cumprod(-1)  
    return F.pad(beta, (0,1), value=1)*F.pad(beta1m_cumprod, (1,0), value=1)
```

► DPMM

```
def model(data):  
    with pyro.plate("beta_plate", T-1):  
        beta = pyro.sample("beta", Beta(1, alpha))  
  
    with pyro.plate("lambda_plate", T):  
        lmbda = pyro.sample("lambda", Gamma(3, 0.05))  
  
    with pyro.plate("data", N):  
        z = pyro.sample("z", Categorical(mix_weights(beta)))  
        pyro.sample("obs", Poisson(lmbda[z]), obs=data)
```

Density estimation: variational distribution

► variational distribution

$$q(\beta, \theta, z) = \prod_{t=1}^{T-1} q_t(\beta_t) \prod_{t=1}^T q_t(\theta_t) \prod_{n=1}^N q_n(z_n)$$

```
def guide(data):
    kappa = pyro.param('kappa', lambda: Uniform(0, 2).sample([T-1]),
                        constraint=constraints.positive)
    tau_0 = pyro.param('tau_0', lambda: Uniform(0, 5).sample([T]),
                        constraint=constraints.positive)
    tau_1 = pyro.param('tau_1', lambda: LogNormal(-1, 1).sample([T]),
                        constraint=constraints.positive)
    phi = pyro.param('phi', lambda: Dirichlet(1/T * torch.ones(T)).sample([N]),
                     constraint=constraints.simplex)

    with pyro.plate("beta_plate", T-1):
        q_beta = pyro.sample("beta", Beta(torch.ones(T-1), kappa))

    with pyro.plate("lambda_plate", T):
        q_lambda = pyro.sample("lambda", Gamma(tau_0, tau_1))

    with pyro.plate("data", N):
        z = pyro.sample("z", Categorical(phi))
```

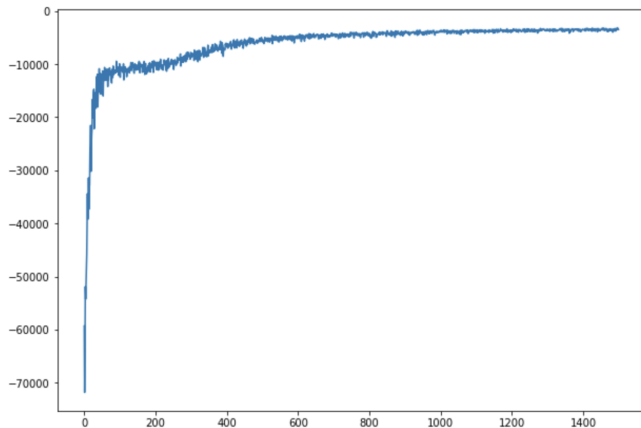
Density estimation: model fitting I

► Stochastic variational inference

```
T = 20
alpha = 1
n_iter = 2000
optim = Adam({"lr": 0.05})
svi = SVI(model, guide, optim, loss=Trace_ELBO())
elbo_values = []

train(n_iter)
```

Density estimation: model fitting II



Density estimation: model fitting III

► Posterior inference

```
samples = torch.arange(0, 300).type(torch.float)

tau0_optimal = pyro.param("tau_0").detach()
tau1_optimal = pyro.param("tau_1").detach()
kappa_optimal = pyro.param("kappa").detach()

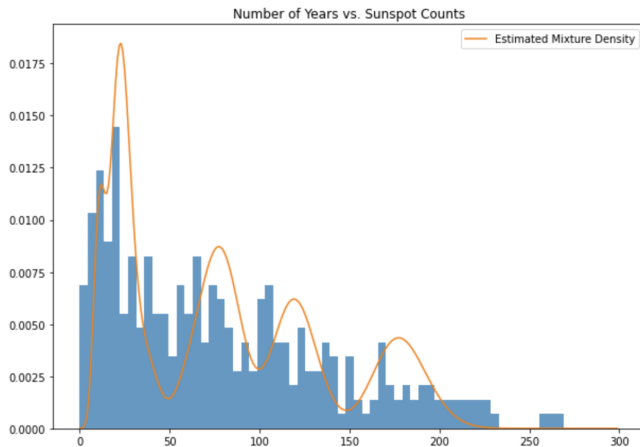
# We make a point-estimate of our latent variables using
# the posterior means of tau and kappa for the cluster params and weights
Bayes_Rates = (tau0_optimal / tau1_optimal)
Bayes_Weights = mix_weights(1. / (1. + kappa_optimal))

def mixture_of_poisson(weights, rates, samples):
    return (weights * Poisson(rates).log_prob(samples.unsqueeze(-1)).exp())

likelihood = mixture_of_poisson(Bayes_Weights, Bayes_Rates, samples)

plt.title("Number of Years vs. Sunspot Counts")
plt.hist(data.numpy(), bins=60, density=True, lw=0, alpha=0.75);
plt.plot(samples, likelihood, label="Estimated Mixture Density")
plt.legend()
plt.show()
```

Density estimation: model fitting IV



Remark and next week

Remark

- ▶ A Dirichlet process is defined as a distribution over random probability measures and used as a prior for a distribution.
- ▶ Dirichlet process mixture models are widely utilized for clustering and density estimation.

Next week

- ▶ Bayesian neural networks

References

- ▶ Pyro tutorials (<https://pyro.ai/examples/>)
- ▶ Murphy (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.